

Plateforme de gestion optique pour Clients et Gestionnaires

1. Présentation du projet

Création d'une application web (et potentiellement mobile) permettant à des **clients** d'acheter des montures de lunettes en ligne à partir d'une ordonnance, et à des **gestionnaires optiques** de suivre, valider et gérer les commandes.

2. Objectifs

- Faciliter la commande de lunettes en ligne pour les clients.
- Permettre aux opticiens de recevoir les ordonnances, émettre des devis (proformas) et gérer les commandes jusqu'à livraison.
- Offrir un système de **suivi en temps réel** pour chaque commande.
- Proposer un mode de **paiement en ligne** ou **en boutique**.
- Fournir un **historique complet** des commandes aux clients et aux gestionnaires.

3. Utilisateurs

3.1. Clients

- Créer un compte / Se connecter / Mot de passe oublié.
- Déposer leur ordonnance (fichier PDF, image, ou prise photo directe) et la carte d'assurance si disponible.
- Choisir une monture à partir d'un modèle de **visualisation 3D**.
- Recevoir une **proforma** (devis) personnalisée.
- Procéder au **paiement en ligne** sécurisé.
- Option de **paiement en boutique**.
- Suivre en temps réel les étapes de leur commande :
 - En attente de devis
 - Devis validé
 - Paiement effectué
 - Commande en cours de livraison
 - Commande livrée
- Accéder à un **historique de commandes**.

3.2. Gestionnaires optiques

- Se connecter à leur espace.
- Suivre toutes les demandes des clients en attente.
- Gérer chaque commande :
 - Visualiser les ordonnances déposées
 - Émettre et envoyer les **proformas** aux clients.
- Marquer les commandes comme **livrées** une fois le traitement terminé.

4. Fonctionnalités détaillées

4.1. Fonctionnalités client

Fonction	Description
Création de compte	Formulaire avec champs : Nom, Prénom, Email, Téléphone, Mot de passe.
Connexion / Mot de passe oublié	Authentification classique, réinitialisation par email.
Dépôt d'ordonnance	Téléchargement de fichier (PDF, JPG, PNG) ou prise de photo.
Choix de monture	Visualisation en 3D des montures (technologie WebGL ou outil intégré).
Réception de la proforma	Notification par email et affichage sur l'espace client.
Paiement en ligne	Intégration de passerelles de paiement (ex : Stripe, PayPal, Paydunya).
Paiement en boutique	Option de réserver et payer directement en boutique.
Suivi de commande	Dashboard montrant chaque étape en temps réel.
Historique des commandes	Liste de toutes les commandes passées, avec détails.

4.2. Fonctionnalités gestionnaire optique

Fonction	Description
Connexion au compte	Authentification sécurisée.
Tableau de bord des demandes	Voir toutes les nouvelles commandes (ordonnances déposées).
Gestion des demandes	Vérifier les ordonnances, créer et envoyer les proformas.
Suivi des commandes	Mettre à jour les statuts : devis validé, paiement reçu, livraison en cours, livraison effectuée.
Historique des commandes clients	Archivage pour chaque client.

5. Architecture technique

- **Back-end** : Django / Django REST Framework (API sécurisée)
- **Front-end Web** : React.js ou Vue.js (recommandé pour visualisation 3D fluide)
- **Front-end mobile** : flutter/react native
- **Base de données** : PostgreSQL / MySQL
- **Stockage des fichiers** : AWS S3, Google Cloud Storage ou serveur dédié
- **Authentification** : JWT (JSON Web Token) ou sessions classiques
- **Paiement** : Stripe, PayPal ou Paydunya (en fonction de ta cible géographique)
- **Notification Email** : SendGrid, Mailgun ou SMTP simple

6. Technologies 3D

- **Three.js** (Javascript) pour la visualisation de modèles 3D.

- Option de permettre à l'utilisateur d'essayer virtuellement les montures (si besoin évolutif).

7. Planning estimatif

Phase	Durée approximative
Rédaction finale du cahier des charges	1 jour
Design des maquettes (UI/UX)	5-7 jours
Développement Back-end (API)	10-15 jours
Développement Front-end client	10-15 jours
Développement espace gestionnaire	5-7 jours
Intégration de la 3D	5-7 jours
Test et débogage	5 jours
Lancement (beta)	2 jours

8. Livrables

- Application Web responsive (ordinateur, tablette, mobile)
- Panel client
- Panel gestionnaire
- API REST documentée
- Documentation technique et utilisateur

9. Contraintes particulières

- **Sécurité élevée** pour les données personnelles et les paiements.
- **Facilité d'utilisation** : Interface claire et intuitive.
- **Performance** : Optimisation pour garantir la rapidité même avec visualisation 3D.

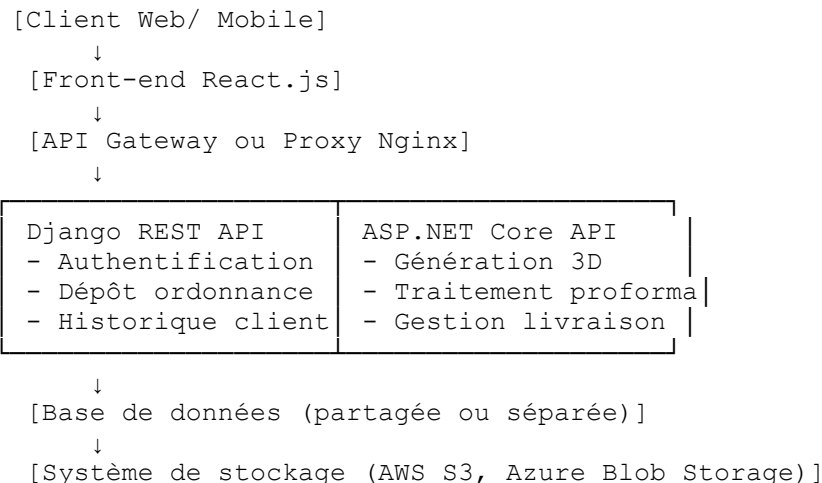
1. Architecture back-end hybride

Technologie	Rôle dans le projet
Django REST Framework (Python)	Gérer les fonctionnalités rapides liées aux utilisateurs : création de comptes, authentification, dépôt d'ordonnances, suivi des commandes, paiement en ligne.
ASP.NET Core Web API (C#)	Gérer la partie lourde/complexe : visualisation 3D, traitement des devis/proformas, gestion avancée du cycle de livraison, interaction poussée avec des bases de données métiers, sécurité renforcée.

2. Communication entre Django REST et ASP.NET

- **API RESTful uniquement** : chaque technologie expose des **endpoints** REST sécurisés.
- **Gestion centralisée** des identifiants utilisateurs pour éviter les incohérences : Django peut être le **serveur principal d'authentification** (SSO possible si besoin).
- **Connexion entre les deux** via des **requêtes HTTP sécurisées** (token d'authentification JWT).

3. Schéma d'architecture simplifié



4. Base de données

Deux options :

- **Option 1** : Base de données unique partagée (ex : PostgreSQL ou Azure SQL Database).

- **Option 2** : Base de données séparées avec synchronisation légère (plus complexe à maintenir, recommandé si besoin d'indépendance forte).

Recommandation : **Base unique partagée** pour ton projet (plus simple, rapide et cohérent).

5. Points forts de ce choix

- ✓ Django pour la rapidité de développement et la gestion facile de tout ce qui est **utilisateur, authentification, upload d'ordonnances**.
 - ✓ ASP.NET pour sa **robustesse**, sa **sécurité** et sa **puissance** sur des traitements métiers complexes (comme émission de devis, gestion de stocks, etc).
 - ✓ Communication via **API REST**, donc facile à maintenir et à faire évoluer vers des microservices si besoin.
-

6. Technologies principales à utiliser

Domaine	Technologie
Front-end	React.js (avec Three.js pour la visualisation 3D)
API Python	Django + Django REST Framework
API C#	ASP.NET Core Web API
Base de données	PostgreSQL ou Azure SQL
Paieement	Stripe ou PayPal API
Stockage fichier	AWS S3 ou Azure Blob Storage
Authentification	Django User + JWT (partagé avec ASP.NET via API)
Proxy/API Gateway	Nginx ou Azure API Management

7. Risques et challenges

- Gestion de la **sécurité** entre deux backends différents.
 - Bien séparer les **responsabilités** pour éviter la duplication de logique.
 - Gestion **des erreurs inter-services** (par exemple si ASP.NET est indisponible).
 - Temps de développement un peu plus long qu'un projet 100% Django ou 100% ASP.NET.
-

8. Plan de déploiement

- Déploiement sur **Azure** ou **AWS**.
- Dockerisation possible pour Django et ASP.NET pour facilité de déploiement.
- Utilisation de **CI/CD pipelines** pour automatiser les mises en production.

Conclusion

Ton idée est très bonne : Django va donner la rapidité et ASP.NET va donner la solidité.

👉 **Je te recommande de garder Django REST Framework** comme **backend principal**, et utiliser **ASP.NET Core** pour les **parties critiques ou lourdes** (par exemple la gestion poussée des devis, logistique complexe, ou visualisation professionnelle).